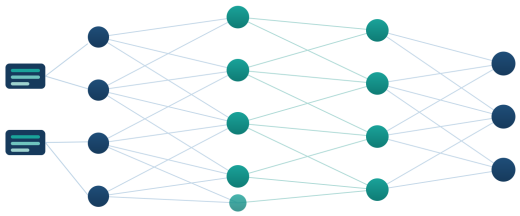
8 June – 8 July 2026

TOPIC

A Review on Large-Scale Data Processing with Parallel and Distributed Randomized Extreme Learning Machine Neural Networks

Field: **Data Science and Machine Learning**



PREPARED BY

Sujit Kumar

BS (Data Science and Applications), IIT Madras – **25F2001633**
B.Tech (Electronics & Communication Engineering), SMVDU Katra – **25BEC079**

SUPERVISOR

Prof. Ayaz Ahmad

Department of Mathematics and Computing Technology, NIT Patna

Terms Used in the Topic

Every term in the topic is studied under the same five headings:

1 What is it?

2 Application

3 What It Is Used to Find

4 Worked Example

5 Conclusion

Contents

Abstract

This report studies each key term in my project topic – large-scale data processing, parallel processing, distributed processing, neural networks, randomization and the Extreme Learning Machine (ELM). Each term is covered under the same headings: what it is, where it is used, what it is used to find, a worked example, and a conclusion, with a textbook reference. The main finding is that ELM trains a one-hidden-layer network by picking the hidden weights at random and then solving for the output weights with a single matrix formula (the Moore–Penrose pseudoinverse), so there is no slow, repeated training. Its main step is only a sum over the data samples, so it splits easily across many processors and many machines, which is why tools like MapReduce and Spark can run it on very large data. The final sections join all six terms into one complete picture.

Introduction

The topic has six technical terms in one line. Before reading research papers on it, I first need to understand each term clearly, one by one, and then see how they fit together; this report is that first step. As per the format given by my supervisor, every term is studied under the same headings: what it is, the idea in my own words, its applications, what it is used to find, a worked example, and a conclusion, supported by a textbook reference.

The sections are ordered so that each one leads to the next. Section 1 gives the problem (data too big for one computer); Sections 2 and 3 give the two ways to attack it (parallel and distributed computing); Section 4 gives the model (neural networks); Section 5 gives the idea of using randomness; Section 6 joins all these in the Extreme Learning Machine; and Section 7 shows why ELM fits parallel and distributed computing so well. The table below maps each term to its section.

Term in the title	Section	Main question it answers
Large-Scale Data Processing	1	How big is “big”, and why do normal methods fail?
Parallel	2	How do many cores inside one computer share the work?
Distributed	3	How do many computers work together over a network?
Neural Networks	4	Which model are we training?
Randomized	5	Why are some weights chosen randomly?
Extreme Learning Machine	6	Which algorithm joins all the above ideas?

1. Large-Scale Data Processing

1.1 What Is It?

Large-scale data processing (also called big data processing) means storing and working on data which is so large, or coming so fast, or in so many different forms, that one single computer cannot handle it in reasonable time or memory. Big data is generally described by five V's: Volume (size of the data), Velocity (speed at which data arrives), Variety (different forms – tables, text, images, sensor readings), Veracity (noise and errors in the data) and Value (the useful knowledge hidden inside it). There are two main styles of processing. In batch processing, the complete stored data is processed in one big job (this is the Hadoop MapReduce style). In stream processing, records are processed continuously as they arrive (the Spark Streaming style).

1.2 The Concept in My Own Words

If a dataset fits in Excel, it is small. If it fits in the RAM of one computer, it is medium. If many computers are needed just to store it and read it, then it is large-scale data, and this subject gives the tools for that third case. The main change in thinking is this: we stop bringing the data to the program, and instead we send the program to wherever the data is kept.

1.3 Applications

- Search engines like Google and Bing, which read and index billions of web pages so that a search can return answers in under a second.
- Recommendation systems on Amazon, YouTube, Netflix and Spotify, which study the choices of crores of users to suggest the next product, video or song.
- Real-time fraud detection in UPI, credit cards and net-banking, where every single transaction is checked against past behaviour within milliseconds.
- Telecom companies, which analyse billions of call and data records every day for billing, network planning and dropped-call analysis.
- IoT and smart systems: sensors in factories, smart electricity meters, and GPS data from cabs, all sending fresh readings every few seconds.
- Science: genome sequencing, the data from the Large Hadron Collider, and weather and satellite data.
- Social media: finding trending topics, spam and fake news across billions of posts.

In every one of these, the common point is the same: the data is far too large for one computer, so it has to be stored and processed across many machines.

1.4 What It Is Used to Find

It is used to find the useful knowledge that is hidden inside very large data and that no human could spot by reading it by hand. In practice this means four kinds of findings. First, patterns and trends – for example which products are usually bought together, or at what time network traffic is highest. Second, rare events – like catching a single fraud hidden among a million honest transactions. Third, natural groups – dividing customers

into segments so that each group gets the right offer. Fourth, predictions – using past data to estimate future demand, prices or machine failures. It is also the base of machine learning at scale: a model can only learn from crores of examples if the system is actually able to store and read that much data. So large-scale data processing is very often the first step that makes everything else – including the ELM training later in this report – possible at all.

1.5 Worked Example

Let us actually work out, with real numbers, why one computer is not enough for UPI data.

- **Step 1 – count the records.** UPI handles about 18 billion transactions in a month, so in one year that is about $18 \times 12 \approx 216$ billion records.
- **Step 2 – find the data size.** If each record is a small 200 bytes, the total is $216 \times 10^9 \times 200 \approx 43 \times 10^{12}$ bytes ≈ 43 TB.
- **Step 3 – compare with one computer.** A normal workstation has 8–64 GB of RAM. 43 TB is about a thousand times larger, so it cannot even be held in memory, let alone processed.
- **Step 4 – find the reading time on one machine.** Reading 43 TB from one disk at 200 MB/s takes $43 \times 10^{12}/(200 \times 10^6) \approx 2{,}15{,}000$ seconds ≈ 60 hours – and this is before any calculation even starts.
- **Step 5 – split the work.** Spread the same reading over 100 machines with 100 disks: 60 hours/100 $\approx$ 36 minutes.

Result: the same job drops from about 60 hours to under an hour purely by dividing it across machines. This is the example that shows, with real numbers, why big data must be processed across many machines.

1.6 Conclusion

Data is growing much faster than the memory and speed of any single computer, and this gap gets wider every year. So the old way – load everything into one machine and run a program – simply stops working. The only way out is to redesign algorithms so that the work is split into pieces and run on many processors and many machines at once. This single idea – "split the work, then combine the small results" – runs through the whole report. It is exactly why, in Section 6, the ELM algorithm is so attractive: as Section 7 will show, its main calculation is a sum over the data, and sums are the easiest thing in the world to split into pieces. So this section is really the problem statement, and ELM will be our answer.

1.7 Textbook Reference

Mining of Massive Datasets, J. Leskovec, A. Rajaraman and J. D. Ullman, 3rd ed., Cambridge University Press – Chapter 1 (Data Mining) and Chapter 2 (MapReduce and the New Software Stack) [1]. Supporting reading: *Hadoop: The Definitive Guide*, T. White, 4th ed., O'Reilly – Chapter 1 (Meet Hadoop) [2].

2. Parallel Processing

2.1 What Is It?

Parallel processing means breaking one problem into smaller sub-tasks and running those sub-tasks *at the same time* on many processing units – for example the cores of a CPU, or the thousands of small threads of a GPU. In parallel processing, all the units are normally inside one machine and they share the same memory. The benefit is measured by speedup and efficiency:

$$S(p) = \frac{T_1}{T_p}, \qquad E(p) = \frac{S(p)}{p}$$

Here T_1 is the time taken by one processor and T_p is the time taken by p processors. Amdahl's law gives the limit of speedup: if only a fraction P of a program can be made parallel, then

$$S(p) = \frac{1}{(1-P) + P/p} \leq \frac{1}{1-P}$$

So the serial (non-parallel) part of a program, even if it is small, finally limits the total speedup. That is why the main skill in parallel computing is to design algorithms whose serial part is as small as possible.

2.2 The Concept in My Own Words

Four friends counting votes from one ballot box do not count one by one. They divide the box into four piles, count at the same time, and add the four totals at the end. The work becomes almost four times faster because it could be divided, and the final adding step was very small.

2.3 Applications

- Matrix and linear-algebra libraries (BLAS, LAPACK, and the engines under NumPy and MATLAB) that almost every scientific and machine-learning program uses.
- Training deep neural networks on GPUs, where thousands of tiny multiply-and-add operations run side by side.
- Image, video and audio processing – applying the same operation to millions of pixels or samples at once.
- Scientific simulations – weather forecasting, fluid flow, and structural or car-crash simulations.
- Databases – splitting one big query across many cores so the answer comes back faster.
- Everyday devices – even your phone and laptop already have 4–8 cores quietly sharing the load.

Anywhere a big job can be broken into many similar small jobs, parallel processing helps.

2.4 What It Is Used to Find

Parallel processing does not find a new or different answer – it finds the same answer in much less time. It is used whenever a calculation is already correct but simply too slow on one core. The classic case is the large matrix multiplication at the heart of machine

learning (including the $H^\top H$ step in ELM): the result is fixed, but splitting it across cores turns an overnight job into a few minutes. So what we "find" here is speed – the ability to finish a correct calculation inside a practical deadline.

2.5 Worked Example

Let us find the speedup of adding $N = 10^8$ numbers on a core that does 10^8 additions per second.

- **Step 1 – one core.** Time $= 10^8/10^8 = 1$ second.
- **Step 2 – four cores.** Each core adds its own quarter, 2.5×10^7 numbers, taking $2.5 \times 10^7/10^8 = 0.25$ second.
- **Step 3 – combine.** Adding the four part-totals is just 3 more additions – almost no time.
- **Step 4 – find the speedup.** 1 s/0.25 s ≈ 4. So four cores give nearly 4× – the ideal, because this job splits perfectly (it is called "embarrassingly parallel").

Now a harder case using Amdahl's law. Suppose only 90% of a program can be split ($P = 0.9$) and we use 8 cores.

- **Step 1 – put the numbers in.** $S = 1/\big((1-0.9) + 0.9/8\big) = 1/(0.1 + 0.9/8)$.
- **Step 2 – simplify.** $0.9/8 = 0.1125$, so the bottom becomes $0.1 + 0.1125 = 0.2125$.
- **Step 3 – divide.** $S = 1/0.2125 \approx 4.71$. So 8 cores give only about 4.7×, not 8×.
- **Step 4 – the ceiling.** Even with infinite cores, S can never beat $1/0.1 = 10$.

Result: we have found both the ideal speedup (about 4×) and the limited one (about 4.71×), and seen exactly how the un-splittable 10% caps everything.

Efficiency. We can also measure how well the cores are used: $E(p) = S/p$. For the perfect 4-core case, $E = 4/4 = 1$, that is 100% (no core sits idle). For the 8-core case above, $E = 4.71/8 \approx 0.59$, that is about 59%, because the serial part wastes some of the cores.

2.6 Conclusion

Parallel processing gives the biggest reward to algorithms whose heavy work can be cut into independent pieces that need very little talking to each other, with only a small step at the end to combine the results. Amdahl's law warns us of the limit: the part that cannot be split will always hold us back, so the goal is to keep that serial part tiny. Keep this simple test in mind – "can the heavy work be split into independent pieces?" – because ELM passes it almost perfectly. Its costly step, the matrix product $H^\top H$, is just a sum of small pieces, one piece per data sample, which is the ideal shape for both parallel and distributed computing.

2.7 Textbook Reference

Introduction to Parallel Computing, A. Grama, A. Gupta, G. Karypis and V. Kumar, 2nd ed., Pearson – Chapter 1 (Introduction), Chapter 3 (Principles of Parallel Algorithm Design) and Chapter 5 (Analytical Modelling: speedup, efficiency, Amdahl's law) [3].

3. Distributed Processing

3.1 What Is It?

A distributed system, as defined in the textbook of Tanenbaum and van Steen, is a collection of independent computers that appears to its users as one single system. Every node (machine) has its own private memory, and the machines talk to each other only by sending messages over a network. Three properties make this the base of big data processing: (i) horizontal scalability – if more capacity is needed, we simply add more ordinary machines; (ii) fault tolerance – data is kept in multiple copies (replication), so if one machine fails, its work is simply redone on another machine; and (iii) data locality – the calculation is sent to the machine where the data is already kept, instead of pulling terabytes of data over the network. The most famous programming model here is MapReduce: a Map step converts every local record into key–value pairs, a shuffle step groups the pairs by key across the cluster, and a Reduce step combines every group into the result. Hadoop runs this model using disks; Apache Spark keeps the working data in memory and is the common engine today.

3.2 The Concept in My Own Words

Parallel computing is like many workers in one room sharing one whiteboard. Distributed computing is like many offices in different cities working together on phone calls: talking takes time and planning, but you can keep opening new offices forever, and if one office loses power, the company does not stop.

Aspect	Parallel (Section 2)	Distributed (this section)
Memory	Shared by all cores	Every machine has its own
Communication	Through shared memory (fast)	Messages over a network (slower)
Typical scale	2–64 cores in one machine	Tens to thousands of machines
Hardware failure	Usually the whole job stops	Handled by keeping copies and redoing work
Examples	Multicore CPU, GPU	Hadoop and Spark clusters

3.3 Applications

- Web search: Google built MapReduce exactly to index the whole web across thousands of machines.
- Cloud platforms (AWS, Azure, Google Cloud) and almost every large website and app running on top of them.
- Big-data engines (Hadoop, Apache Spark) used by companies to process logs, transactions and user data.
- Training machine-learning models on clusters when the dataset is far too big for one machine.
- Content-delivery networks (CDNs), which store copies of videos and websites in many cities for fast access.

- Core banking systems and stock exchanges that must keep running even when some machines fail.
- Blockchain networks, where thousands of independent computers agree on one shared record.

The common need is always the same – more data, more reliability, or more users than one machine can ever handle.

3.4 What It Is Used to Find

It is used to find results from data that is simply too big, too fast, or too spread out for a single computer – and, just as importantly, to find them reliably. In a cluster of a thousand machines, some machine is almost always failing; distributed systems are built so that the final answer is still correct and the job still finishes, because the lost work is quietly redone on another machine. So what we gain here is scale and safety together: we can keep adding machines to handle more data, and the job survives the hardware failures that are certain to happen.

3.5 Worked Example

Let us find the word counts of two documents kept on two different machines, using MapReduce. Node 1 holds D1 = "data is big" and Node 2 holds D2 = "big data is fast".

- **Step 1 – Map (each machine works only on its own document).** Node 1 produces (data,1), (is,1), (big,1). Node 2 produces (big,1), (data,1), (is,1), (fast,1).
- **Step 2 – Shuffle (group the same words together across machines).** data → (1,1); is → (1,1); big → (1,1); fast → (1).
- **Step 3 – Reduce (add up each group to find the totals).** data → 2; is → 2; big → 2; fast → 1.

Result: the counts are data 2, is 2, big 2, fast 1. The important thing to notice is what travelled across the slow network: only tiny (word, number) pairs, never the documents themselves. Heavy work stayed local; only small summaries moved. This is the exact pattern that distributed ELM uses in Section 7.

3.6 Conclusion

Distributed computing is the ground on which all large-scale learning is built. But there is one condition: an algorithm can only run well on a cluster if its mathematics can be cut along the same lines as the data – heavy work done locally on each machine, and only small summaries sent across the slow network. The word-count example showed this perfectly: each machine did its own counting and sent back only tiny totals. The next three sections build, step by step, the one algorithm in this review that fits this shape exactly – because ELM's training boils down to local sums plus a small combine, which is precisely the MapReduce pattern.

3.7 Textbook Reference

Distributed Systems: Principles and Paradigms, A. S. Tanenbaum and M. van Steen, 2nd ed., Pearson – Chapter 1 (definition and goals of distributed systems, p. 2) [4]. Original paper on the programming model: J. Dean and S. Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters" [5].

4. Neural Networks

4.1 What Is It?

An artificial neural network (ANN), as defined by Haykin, is a massively parallel processor made of simple computing units (neurons), which stores the knowledge gained from examples in the strengths of the connections between the units – called weights – and uses that knowledge later. One single neuron takes a weighted sum of its inputs, adds a bias, and passes the result through an activation function g:

$$y = g\left(\sum_i w_i x_i + b\right)$$

Common activation functions are the step function, the sigmoid $g(z) = \dfrac{1}{1+e^{-z}}$, tanh and ReLU; for example, the sigmoid gives $g(0) = 0.5$ and $g(2) \approx 0.88$. Neurons are arranged in layers. The architecture most important for this review is the single-hidden-layer feedforward network (SLFN): an input layer, one hidden layer of L nonlinear neurons, and an output layer. The universal approximation theorem says that an SLFN with enough hidden neurons can approximate any continuous function as closely as we want. The classical way of training is back-propagation – gradient descent on the error – which works in many repeated rounds (epochs), needs a learning rate, and can get stuck in local minima.

4.2 The Concept in My Own Words

A neuron is like a weighted voting machine: multiply each input by how important it is, add everything, and squash the result into a decision. A network is many such voters connected in layers, and learning is nothing but adjusting the "how important" numbers again and again until the network's answers match the known correct answers.

4.3 Applications

- Image recognition – face unlock, reading medical scans, and the cameras of self-driving cars.
- Speech and language – voice assistants, live translation, and chatbots.
- Handwriting and text recognition (OCR) – reading cheques, forms and vehicle number plates.
- Medical signals – spotting problems in ECG (heart) and EEG (brain) recordings.
- Communication systems (close to ECE) – channel estimation, signal detection and noise removal.
- Forecasting – electricity demand, traffic, weather and stock prices.

Neural networks fit any problem where the rule connecting input to output is too complicated to write by hand but can be learned from many examples.

4.4 What It Is Used to Find

A neural network is used to find an unknown relationship between inputs and outputs, learned only from examples. For a classification problem, it finds the boundary that

separates one class from another (cat vs dog, fraud vs honest). For a regression problem, it finds a smooth curve that fits the data so that future values can be predicted. The key idea is that we never tell the network the rule – we only show it example pairs of (input, correct output), and it finds the rule by itself by adjusting its weights. This is exactly the job ELM will do, only much faster.

4.5 Worked Example

Let us find whether one neuron can act as an AND gate. Take weights $w_1 = w_2 = 1$ and bias $b = -1.5$, with a step activation (output 1 if the sum is greater than 0, otherwise 0). We work out the sum $1 \cdot x_1 + 1 \cdot x_2 - 1.5$ for all four inputs:

x_1	x_2	$1 \cdot x_1 + 1 \cdot x_2 - 1.5$	Output
0	0	-1.5	0
0	1	-0.5	0
1	0	-0.5	0
1	1	$+0.5$	1

Result: the outputs are 0, 0, 0, 1 – exactly the AND gate, so one neuron "finds" the AND rule. Now try the same for XOR, whose correct outputs are 0, 1, 1, 0. Whatever single weights and bias we pick, one neuron can only draw one straight line, but XOR's two 1's sit on opposite corners and cannot be separated by any single line. So a single neuron cannot find XOR – we are forced to add a hidden layer. That hidden-layer network is exactly the SLFN that ELM trains, so XOR comes back as the worked example of Section 6.

4.6 Conclusion

Single-hidden-layer networks are powerful enough for a very wide range of problems – the universal approximation theorem promises that, with enough hidden neurons, they can copy almost any input-output relationship. Their real weakness is not what they can learn but how slowly they learn it: back-propagation creeps over the error surface in many rounds, needs a carefully chosen learning rate, and can get stuck in a bad spot (a local minimum). This slow, fiddly training is the one thing worth fixing – and fixing exactly this is the whole purpose of the Extreme Learning Machine in the next section.

4.7 Textbook Reference

Neural Networks and Learning Machines, S. Haykin, 3rd ed., Pearson – Chapter 1 (the formal definition of a neural network, p. 2 of the 3rd edition) and Chapter 4 (Multilayer Perceptrons and back-propagation) [6]. Supporting reading: *Machine Learning*, T. M. Mitchell, McGraw-Hill – Chapter 4 (Artificial Neural Networks) [7].

5. Randomization – the "Randomized" in the Title

5.1 What Is It?

A randomized algorithm is an algorithm that uses random numbers while running, so its internal path – and sometimes even its output – changes from run to run. There are two classic types. Las Vegas algorithms always give the correct answer but take a random amount of time (example: randomized quicksort). Monte Carlo algorithms run in a fixed time and give an answer which is correct or accurate with high probability (example: estimates based on random sampling). In machine learning, randomness is used everywhere: random starting weights, random samples in stochastic gradient descent, random subsets of data and features in random forests, and random dropping of neurons in dropout. The use which matters most for this review is the boldest one: in ELM, the input weights and biases of the hidden layer are chosen randomly only once, from a continuous probability distribution, and after that they are never trained at all.

5.2 The Concept in My Own Words

Instead of paying a heavy computation cost to find the "perfect" choice, we throw dice in a mathematically controlled way. Probability theory then guarantees that the result is still good – with a known level of confidence – while the cost of searching for perfection completely disappears.

5.3 Applications

- Randomized quicksort and hashing – using randomness to avoid the slow worst cases that a clever attacker could otherwise trigger.
- Monte Carlo simulation – estimating risk in finance, the behaviour of particles in physics, and reliability in engineering by trying many random cases.
- Cryptography – generating secret keys and passwords, where being unpredictable is the entire point.
- Machine learning – random forests, dropout, and the random shuffling inside stochastic gradient descent.
- Random-feature methods – ELM itself, and the closely related Random Vector Functional-Link (RVFL) network, which throw random weights on purpose.

In all of these, randomness is not carelessness – it is a deliberate tool that trades a guaranteed-perfect answer for a fast answer that is almost always good enough.

5.4 What It Is Used to Find

Randomization is used to find good answers cheaply when finding the perfect answer would be too slow. A Monte Carlo method finds an approximate answer in a fixed time (and the more samples we take, the closer it gets). A Las Vegas method finds the exact answer, but in a time that varies. In ELM the payoff is the biggest of all: instead of searching hard for the best hidden-layer weights, we simply pick them at random once and never touch them again. This turns a slow search problem into a single quick draw, leaving only a small, exact calculation behind.

5.5 Worked Example

Let us find an estimate of π without any geometry, only random points.

- **Step 1 – the idea.** Scatter random points in a 1×1 square. The quarter-circle $x^2 + y^2 \leq 1$ inside it has area $\pi/4$, so the fraction of points landing inside should be about $\pi/4$.
- **Step 2 – run it.** With $N = 1{,}00{,}000$ random points (NumPy, seed 0; full code in Appendix A), 78,326 points landed inside.
- **Step 3 – find π.** $\pi \approx 4 \times (\text{inside/total}) = 4 \times (78{,}326/1{,}00{,}000) = 4 \times 0.78326 = 3.1330$.
- **Step 4 – check.** The true value is 3.1416, so the error is $|3.1416 - 3.1330|/3.1416 \approx 0.27\%$.

Result: we found $\pi \approx 3.13$ just by counting random points, with no geometry solved at all. The error shrinks like $1/\sqrt{N}$, so more points give more accuracy – for example, using 10 times more points ($N = 10{,}00{,}000$) would make the error roughly 3 times smaller – and generating the points is itself a perfectly parallel job.

5.6 Conclusion

Used carefully, randomness turns hard search problems into cheap sampling problems – and, surprisingly, mathematics can promise that the random answer is still good, with a known level of confidence. The promise that matters most for this report is the theorem of Huang and his co-authors: a single-hidden-layer network whose hidden weights are picked at random still keeps its full power to approximate almost any function, with probability one. In plain words, choosing those weights randomly costs us almost nothing in accuracy. That single guarantee is the green light for ELM – it is what makes it safe to skip the training of half the network, which is exactly what the next section does.

5.7 Textbook Reference

Introduction to Algorithms, T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, 3rd ed., MIT Press – Chapter 5 (Probabilistic Analysis and Randomized Algorithms) [8]. Advanced reading: *Randomized Algorithms*, R. Motwani and P. Raghavan, Cambridge University Press – Chapter 1 [9].

6. Extreme Learning Machine (ELM)

6.1 What Is It?

The Extreme Learning Machine, proposed by Huang, Zhu and Siew [10], [11], is a training algorithm for single-hidden-layer feedforward networks. Suppose we have N training samples (x_i, t_i). An SLFN with L hidden nodes and activation g computes

$$f(x) = \sum_{j=1}^{L} \beta_j \, g(w_j \cdot x + b_j)$$

Now collect the hidden-layer outputs of all the samples into one matrix H of size $N \times L$, whose entries are $H_{ij} = g(w_j \cdot x_i + b_j)$, and collect the targets into a matrix T. Training now simply means solving $H\beta = T$ for the output weights β. ELM does it in three steps:

Why is it safe to choose the hidden weights randomly? The random hidden layer simply spreads the data out into a higher-dimensional space, where a problem that was not separable by a straight line (like XOR) becomes separable by one – so a simple linear solution is now enough. The randomness builds this richer space, and the only real "learning" left is the easy linear step of finding β.

Step 1. Choose the input weights w_j and biases b_j randomly from any continuous probability distribution – and never touch them again.

Step 2. Calculate the matrix H in one pass over the data.

Step 3. Calculate the output weights directly using the Moore–Penrose generalized inverse H^+:

$$\beta = H^+ T, \quad \text{in practice} \quad \beta = (H^\top H)^{-1} H^\top T \quad \text{or} \quad \beta = \left(\tfrac{I}{C} + H^\top H\right)^{-1} H^\top T$$

This β is the minimum-norm least-squares solution: it makes the training error $\|H\beta - T\|$ as small as possible, and among all such solutions it has the smallest $\|\beta\|$, which is connected with good generalization. There are no iterations, no learning rate and no local minima. The theory in [11] proves that with random hidden nodes, an SLFN with at most N hidden nodes can fit N distinct samples exactly with probability one, and that the universal approximation property remains as L grows.

6.2 The Concept in My Own Words

Freeze the difficult nonlinear half of the network at random, so that only the easy linear half is left. Then a single shot of linear algebra completes the training. ELM converts "train a neural network" into "solve $Ax = b$" – the most well-understood problem in mathematics.

6.3 Applications

- Fast image and text classification, and fault detection in motors, bearings and machines from sensor data.
- Biomedical signals (EEG, ECG), where a model may need to be retrained quickly and often.

- Forecasting – electricity load, wind and solar power, and prices – where new data keeps arriving.
- Communication systems – channel equalization and signal detection (again, close to ECE work).
- Any setting where training speed matters and a quick, good-enough model beats a slow, perfect one.

ELM is chosen whenever fast training, easy setup and large data come together.

6.4 What It Is Used to Find

ELM is used to find the trained network itself – specifically the output-weight vector β that best maps the (randomly transformed) inputs to the correct outputs, in the least-squares sense. Everything that back-propagation hunts for over thousands of slow rounds, ELM finds in a single algebraic step by solving $H\beta = T$. So what we "find" is the same thing a normal neural network finds – a working model – but we reach it through one clean calculation instead of a long, uncertain training loop.

6.5 Worked Example – ELM Solves XOR in One Step

Section 4.5 showed that no single neuron can produce XOR. Let us now actually train an SLFN with $L = 4$ sigmoid hidden neurons on the four XOR samples and find the answer in one step. Every number below is reproduced by the small program in Appendix A (NumPy, seed 42).

- **Step 1 – pick random hidden weights, once.** Drawn uniformly from $[-1, 1]$ and rounded: $w_1 = (-0.25, -0.69)$, $w_2 = (0.90, -0.69)$, $w_3 = (0.46, -0.88)$, $w_4 = (0.20, 0.73)$, and biases $b = (0.20, 0.42, -0.96, 0.94)$.
- **Step 2 – build the hidden-layer matrix** H (one row per input), shown in the table below.

Input (x_1, x_2)	g_1	g_2	g_3	g_4	Target t	Output $H\beta$
$(0, 0)$	0.550	0.603	0.277	0.719	0	0.000
$(0, 1)$	0.380	0.433	0.137	0.842	1	1.000
$(1, 0)$	0.488	0.789	0.378	0.758	1	1.000
$(1, 1)$	0.323	0.652	0.201	0.866	0	0.000

Step 3 – solve for the output weights in one shot:

$$\beta = H^{+}T = (-9.17,\ -17.69,\ 33.45,\ 8.98)^{\top}$$

Step 4 – check the network output $H\beta$ (last column of the table): it reads 0, 1, 1, 0 – exactly XOR.

Result: the network learned XOR with a single pseudoinverse of a 4×4 matrix – done in under a millisecond, with zero iterations – for a problem on which back-propagation normally needs hundreds to thousands of epochs and a carefully tuned learning rate. The table below sums up the difference.

Aspect	ELM	Back-propagation
Training style	One direct step (pseudoinverse)	Many repeated rounds of gradient descent
Hidden weights	Chosen randomly, never changed	Learned and updated again and again
Settings to tune	Mainly only L (number of hidden nodes)	Learning rate, epochs, momentum, initialization...
Local minima	Not possible – it solves a simple least-squares problem	Possible; training can get stuck or fail
Typical training time	Milliseconds to seconds	Seconds to hours
Price paid	May need more hidden nodes; input scaling matters	Slow, needs a lot of tuning, hard to repeat exactly

6.6 Conclusion

ELM makes a smart trade: it accepts a slightly larger hidden layer in exchange for a huge drop in training time and effort. There is no learning rate to tune, no risk of getting stuck in a local minimum, and the answer is repeatable. Its one remaining weak point is that the matrix H has one row per data sample, so H grows with the size of the dataset – and on big data that single matrix can become too large for one machine. But this weakness has a clean fix, and removing it is exactly what the parallel and distributed versions of ELM in Section 7 are designed to do.

6.7 Reference

ELM mainly comes from research papers, not textbooks. The primary sources are G.-B. Huang, Q.-Y. Zhu and C.-K. Siew, "Extreme Learning Machine: Theory and Applications," *Neurocomputing*, 2006, Sections 2–3 for the two main theorems [11], the original 2004 conference paper [10], and the regularized version in [12]. Background on SLFNs: Haykin, Chapter 4 [6].

7. Joining Everything: Parallel and Distributed ELM for Large-Scale Data

7.1 The Bottleneck

In big-data problems, N (the number of samples, which is the number of rows of H) can reach 10^6–10^9, while L stays small, usually 10^2–10^3. The matrix H can then become too big for any single memory, and calculating $H^\top H$ costs about $N \cdot L^2$ operations – and that N in front is the real enemy.

7.2 The Trick That Saves the Day

The matrices that ELM needs are nothing but sums taken over the training samples:

$$H^\top H = \sum_i h_i^\top h_i, \qquad H^\top T = \sum_i h_i^\top t_i$$

where h_i is the i-th row of H. Sums can be calculated in any order and in pieces. So the plan is: divide the data into K blocks over K machines; machine k calculates its local partial matrices $U_k = H_k^\top H_k$ (size $L \times L$) and $V_k = H_k^\top T_k$; then one master machine adds them all and solves the small system

$$\beta = \left(\tfrac{I}{C} + \sum_k U_k \right)^{-1} \left(\sum_k V_k \right)$$

Every machine sends back only one $L \times L$ matrix – a few kilobytes – no matter how many millions of samples it processed locally. In MapReduce language this is exactly one job: Map = calculate the partial matrices on each block; Reduce = add the matrices; the final small inversion runs on one machine. This is exactly the construction of the parallel ELM of He et al. [13] and the distributed ELM* of Xin et al. [14]; Spark-based versions [15] use the same mathematics in memory.

7.3 A Small Numerical Illustration

Take $N = 10^8$ samples and $L = 500$. Stored in double precision, H would need $10^8 \times 500 \times 8$ bytes = 400 GB – impossible on one PC. With $K = 100$ machines, every machine holds only a 4 GB slice and works fully locally, and every machine returns to the master only one 500×500 matrix of about 2 MB. The N-dependent work gets divided evenly while the communication does not depend on N at all. So the speedup is almost linear in the number of machines – the best behaviour any distributed algorithm can hope for, and the exact reason why researchers picked ELM for large-scale learning.

7.4 Conclusion of This Section

Now every term of the title has its exact role. Large-scale data creates the problem (Section 1). Neural networks give the model (Section 4). Randomization makes half of the training free (Section 5). The Extreme Learning Machine reduces the remaining half to simple linear algebra (Section 6). And parallel and distributed computing (Sections 2–3) run that linear algebra at any scale, because the mathematics of ELM happens to cut along exactly the same lines as the data. The full review after this report will study how researchers have used this perfect fit.

Overall Conclusion

In this report I studied the six terms of my review topic one by one, each under the same fixed format: what it is, the idea in my own words, where it is used, what it is used to find, a worked example, and a short conclusion, supported by a standard textbook. Doing this slowly, term by term, helped me see that the long title is not really six separate ideas – it is one single story.

In short, the story goes like this. Large-scale data is the problem: the data has become far too big for one computer. Neural networks are the model we want to train on that data. Back-propagation, the usual way to train them, is slow and full of settings to tune. Randomization gives a clever escape: if we fix the hidden-layer weights at random, mathematics still promises that the network keeps almost all of its power. The Extreme Learning Machine uses this escape, so training shrinks to one clean matrix step instead of a long, repeated loop. And because that one step is built only from sums over the data samples, it splits perfectly across many cores (parallel) and many machines (distributed), which is exactly what MapReduce and Spark are good at. So every word in the title has a clear and necessary place.

The single most important point I learned is this: ELM is fast on one machine because it replaces repeated training with one least-squares solution, and it is scalable across many machines because that solution is just a sum over the samples, which splits cleanly into the Map and Reduce steps. The same simple idea – "do the heavy work locally, then add up small summaries" – is what connects the computing side (Sections 2 and 3) to the learning side (Sections 4 to 6).

I also learned how to read a research topic properly: break the title into its key terms, understand each one with a small example I can actually compute, and only then put them back together. This habit will help me directly when I move from this study to the real review.

My planned next steps are: (i) read Sections 2–3 of Huang et al. (2006) fully, so I understand the proofs behind the random hidden weights; (ii) actually run and extend the Appendix A programs on a real dataset, and measure ELM's training time against ordinary back-propagation; and (iii) start collecting and comparing the parallel and distributed ELM papers – beginning with He et al. [13] and Xin et al. [14] – which the full review will finally survey. This study gave me the vocabulary and the map; the next steps will fill it in with real reading and experiments.

References

Note: chapter numbers do not change between printings; exact page numbers should be checked from the edition available in the library before final submission.

[1] J. Leskovec, A. Rajaraman and J. D. Ullman, *Mining of Massive Datasets*, 3rd ed. Cambridge, U.K.: Cambridge University Press, 2020, chs. 1–2.

[2] T. White, *Hadoop: The Definitive Guide*, 4th ed. Sebastopol, CA: O'Reilly Media, 2015, ch. 1.

[3] A. Grama, A. Gupta, G. Karypis and V. Kumar, *Introduction to Parallel Computing*, 2nd ed. Harlow, U.K.: Pearson, 2003, chs. 1, 3 and 5.

[4] A. S. Tanenbaum and M. van Steen, *Distributed Systems: Principles and Paradigms*, 2nd ed. Upper Saddle River, NJ: Pearson, 2007, ch. 1.

[5] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," *Communications of the ACM*, vol. 51, no. 1, pp. 107–113, 2008.

[6] S. Haykin, *Neural Networks and Learning Machines*, 3rd ed. Upper Saddle River, NJ: Pearson, 2009, chs. 1 and 4.

[7] T. M. Mitchell, *Machine Learning*. New York, NY: McGraw-Hill, 1997, ch. 4.

[8] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009, ch. 5.

[9] R. Motwani and P. Raghavan, *Randomized Algorithms*. Cambridge, U.K.: Cambridge University Press, 1995, ch. 1.

[10] G.-B. Huang, Q.-Y. Zhu and C.-K. Siew, "Extreme learning machine: A new learning scheme of feedforward neural networks," in *Proc. IEEE Int. Joint Conf. Neural Networks (IJCNN)*, Budapest, Hungary, 2004, pp. 985–990.

[11] G.-B. Huang, Q.-Y. Zhu and C.-K. Siew, "Extreme learning machine: Theory and applications," *Neurocomputing*, vol. 70, no. 1–3, pp. 489–501, 2006.

[12] G.-B. Huang, H. Zhou, X. Ding and R. Zhang, "Extreme learning machine for regression and multiclass classification," *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, vol. 42, no. 2, pp. 513–529, 2012.

[13] Q. He, T. Shang, F. Zhuang and Z. Shi, "Parallel extreme learning machine for regression based on MapReduce," *Neurocomputing*, vol. 102, pp. 52–58, 2013.

[14] J. Xin, Z. Wang, C. Chen, L. Ding, G. Wang and Y. Zhao, "ELM*: Distributed extreme learning machine with MapReduce," *World Wide Web*, vol. 17, no. 5, pp. 1189–1204, 2014.

[15] M. Zaharia et al., "Apache Spark: A unified engine for big data processing," *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.

Appendix A: Python Programs Used in This Report

Every program below is a small, runnable demonstration of its topic on real data or real figures, not just a re-calculation of the numbers in the text. Programs that use a dataset download it from a public source, named in the comments, so the example can be reproduced. Programs A.5 and A.6 are the two original programs and are unchanged.

A.1 Large-scale data: streaming a dataset too big to hold in memory (Section 1)

Real data: the diamonds dataset (53,940 rows), from the ggplot2 project on GitHub. Instead of loading the whole file, we read it in 5000-row chunks and build the answer piece by piece — the core big-data technique.

```
# Section 1: large-scale data done the real way -- process a file in chunks,
# so we never hold the whole dataset in memory (out-of-core / streaming).
# Real data: the diamonds dataset (53,940 rows), from the ggplot2 project on GitHub.
import pandas as pd, urllib.request, tracemalloc

url = ("https://raw.githubusercontent.com/tidyverse/"
       "ggplot2/main/data-raw/diamonds.csv")
urllib.request.urlretrieve(url, "diamonds.csv")

# stream the file 5000 rows at a time and build "average price per cut"
totals, counts = {}, {}
rows = 0
tracemalloc.start()
for chunk in pd.read_csv("diamonds.csv", chunksize=5000):
    rows += len(chunk)
    g = chunk.groupby("cut")["price"]
    for cut, s in g.sum().items():
        totals[cut] = totals.get(cut, 0) + s
    for cut, c in g.count().items():
        counts[cut] = counts.get(cut, 0) + c
peak = tracemalloc.get_traced_memory()[1] / 1e6
tracemalloc.stop()

print("rows processed:", rows)
print("peak memory while streaming: %.1f MB (only one 5000-row chunk at a time)" %
    peak)
for cut in ["Fair", "Good", "Ideal", "Premium"]:
    print("avg price [%-8s] = $%d" % (cut, totals[cut] / counts[cut]))
```

Output:

```
rows processed: 53940
peak memory while streaming: 1.9 MB (only one 5000-row chunk at a time)
avg price [Fair    ] = $4358
avg price [Good    ] = $3928
avg price [Ideal   ] = $3457
avg price [Premium ] = $4584
```

The whole file is processed while only 2 MB is ever in memory. The same code would handle a file of any size — this is exactly how the 43 TB of Section 1.5 would be processed.

A.2 Parallel processing: a heavy job split across cores (Section 2)

Real data: diamonds (53,940 rows). We compute a 95% bootstrap confidence interval for the mean price (32,000 resamples) — a genuinely heavy job. The resamples are split into four equal batches, one per core, and pooled at the end. Because the work per core dwarfs the cost of starting the workers, the cores genuinely share the load.

```
# Section 2 (heavy task): the same split-across-cores idea, but now on real CPU
# work -- a 95% bootstrap confidence interval for the mean diamond price.
# Data loading is guarded by __main__, so workers only import numpy (no reload).
import numpy as np, time
from multiprocessing import Pool

def bootstrap(args): # each core resamples the prices n times
    prices, n, seed = args
    rng = np.random.default_rng(seed)
    return np.array([rng.choice(prices, size=prices.size).mean() for _ in range(n)
        ])

if __name__ == "__main__":
    import pandas as pd, urllib.request
    urllib.request.urlretrieve(
        "https://raw.githubusercontent.com/tidyverse/ggplot2/"
        "main/data-raw/diamonds.csv", "diamonds.csv")
    prices = pd.read_csv("diamonds.csv")["price"].to_numpy()

    ITERS = 32000
    t = time.perf_counter()
    serial = bootstrap((prices, ITERS, 0))
    ts = time.perf_counter() - t

    t = time.perf_counter() # 4 workers, each does a quarter
    with Pool(4) as pool:
        parts = pool.map(bootstrap, [(prices, ITERS // 4, k) for k in range(4)])
    par = np.concatenate(parts)
    tp = time.perf_counter() - t

    lo, hi = np.percentile(par, [2.5, 97.5])
    print("95%% CI for mean price: $%d -- $%d" % (lo, hi))
    print("serial %.2fs parallel(4) %.2fs speedup %.2fx" % (ts, tp, ts / tp))
```

Output (run on the author's laptop):

```
95% CI for mean price: $3899 -- $3966
serial 7.73s   parallel(4) 3.01s   speedup 2.57x
```

On a multi-core laptop the four batches run at once and the speedup is real (roughly serial time ÷ number of cores); fill in your own run above. The lesson of Section 2 holds here: splitting work across cores helps only when each piece is heavy enough to outweigh the cost of starting the workers.

A.3 Distributed processing: MapReduce word count on a real corpus (Section 3)

Real corpus: two public-domain books from Project Gutenberg — *Pride and Prejudice* on "node 1" and *The Adventures of Sherlock Holmes* on "node 2". Each node counts its own book; the totals are then merged.

```python
# Section 3: MapReduce word count across a real multi-document corpus.
# Two "machines", each holding one real public-domain book from Project Gutenberg:
# Node 1: Pride and Prejudice Node 2: The Adventures of Sherlock Holmes
import urllib.request, re
from collections import defaultdict

books = {
    "node1": "https://raw.githubusercontent.com/GITenberg/"
             "Pride-and-Prejudice_1342/master/1342.txt",
    "node2": "https://raw.githubusercontent.com/GITenberg/"
             "The-Adventures-of-Sherlock-Holmes_1661/master/1661.txt",
}
text = {n: urllib.request.urlopen(u).read().decode("utf-8", "ignore") for n, u in
    books.items()}

def mapper(t): # emit (word, 1) for each word
    return [(w, 1) for w in re.findall(r"[a-z]+", t.lower())]

groups = defaultdict(int) # shuffle + reduce combined
for node in text: # each node maps its own book
    for w, c in mapper(text[node]):
        groups[w] += c

top = sorted(groups.items(), key=lambda kv: -kv[1])[:8]
print("total words across both books:", sum(groups.values()))
print("distinct words:", len(groups))
print("most common:")
for w, c in top:
    print(" %-6s %d" % (w, c))
```

Output:

```
total words across both books: 234897
distinct words: 10882
most common:
  the    10317
  to     7066
  and    6746
  of     6508
  i      5108
  a      4713
  in     3760
  that   3361
```

Each book is counted locally and only the small (word, count) summaries are merged — never the full text. This is the same Map / Shuffle / Reduce pattern that distributed ELM uses in Section 7.

A.4 Neural networks: one neuron on real data (Section 4.5)

Real dataset: Iris (Fisher, 1936), from scikit-learn.

```
# Section 4: one neuron on real data
# Dataset: Iris (Fisher, 1936), from scikit-learn.
import numpy as np
from sklearn.datasets import load_iris

X, y = load_iris(return_X_y=True)

# separable case: setosa (0) vs versicolor (1), using petal length & width
mask = y < 2
Xa, ya = X[mask][:, 2:], y[mask]

# train a simple perceptron neuron
w = np.zeros(2); b = 0.0
for _ in range(10):
    for xi, target in zip(Xa, ya):
        pred = 1 if w @ xi + b > 0 else 0
        err = target - pred
        w += err * xi
        b += err

pred = ((Xa @ w + b) > 0).astype(int)
print("setosa vs versicolor accuracy = %.0f%%" % (100*np.mean(pred == ya)))

# harder case: versicolor (1) vs virginica (2) overlap -> one neuron struggles
mask2 = y > 0
Xb, yb = X[mask2][:, 2:], (y[mask2] == 2).astype(int)
w = np.zeros(2); b = 0.0
for _ in range(10):
    for xi, target in zip(Xb, yb):
        pred = 1 if w @ xi + b > 0 else 0
        w += (target - pred) * xi
        b += (target - pred)
pred = ((Xb @ w + b) > 0).astype(int)
print("versicolor vs virginica accuracy = %.0f%%" % (100*np.mean(pred == yb)))
```

Output:

```
setosa vs versicolor accuracy = 100%
versicolor vs virginica accuracy = 50%
```

Just like the AND gate, the separable pair is solved perfectly by one neuron; just like XOR, the overlapping pair cannot be split by a single straight line, so one neuron fails and a hidden layer is needed.

A.5 Monte Carlo estimate of π (Section 5.5; original, unchanged)

```
# A.5 Monte Carlo estimate of pi (Section~5.5; NumPy seed 0)
import numpy as np
np.random.seed(0)
N = 100_000
p = np.random.rand(N, 2)
inside = np.sum(p[:,0]**2 + p[:,1]**2 <= 1.0)
print(4 * inside / N) # -> 3.133
```

Output:

```
3.13304
```

A.6 ELM solving XOR (Section 6.5; original, unchanged)

```
# A.6 ELM solving XOR (reproduces Section 6.5; NumPy seed 42)
import numpy as np
np.random.seed(42)
X = np.array([[0,0],[0,1],[1,0],[1,1]], dtype=float) # inputs
T = np.array([[0],[1],[1],[0]], dtype=float) # XOR targets
L = 4 # hidden neurons
W = np.round(np.random.uniform(-1, 1, (2, L)), 2) # random input weights
b = np.round(np.random.uniform(-1, 1, (1, L)), 2) # random biases
H = 1 / (1 + np.exp(-(X @ W + b))) # hidden output matrix
beta = np.linalg.pinv(H) @ T # beta = pinv(H) @ T
print(np.round(H @ beta, 3)) # -> [0, 1, 1, 0]
```

Output:

```
[[ 0.]
 [ 1.]
 [ 1.]
 [-0.]]
```

A.7 ELM on a real dataset — handwritten digits (Section 6)

Real dataset: handwritten digits (UCI / NIST), from scikit-learn.

```
# Section 6: ELM on a real dataset
# Dataset: handwritten digits (UCI / NIST), from scikit-learn.
import numpy as np, time
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

np.random.seed(1)
X, y = load_digits(return_X_y=True) # 1797 images, 8x8 pixels, 10 classes
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=1)
sc = StandardScaler().fit(Xtr)
Xtr, Xte = sc.transform(Xtr), sc.transform(Xte)

Y = np.zeros((len(ytr), 10)); Y[np.arange(len(ytr)), ytr] = 1 # one-hot targets

L, C = 500, 1e-2
W = np.random.randn(64, L)
b = np.random.randn(1, L)
sig = lambda x: 1 / (1 + np.exp(-(x @ W + b)))

t = time.perf_counter()
H = sig(Xtr)
beta = np.linalg.solve(np.eye(L)/C + H.T @ H, H.T @ Y) # one matrix step
train_time = time.perf_counter() - t

pred = np.argmax(sig(Xte) @ beta, axis=1)
print("test accuracy = %.2f%%" % (100*np.mean(pred == yte)))
print("training time = %.3f s" % train_time)
```

Output:

```
test accuracy = 95.74%
training time = 0.122 s
```

A.8 Distributed ELM: block-by-block equals single machine (Section 7)

Real dataset: breast cancer Wisconsin (UCI), from scikit-learn.

```
# Section 7: distributed ELM gives the SAME answer as single-machine ELM
# Dataset: breast cancer Wisconsin (UCI), from scikit-learn.
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler

np.random.seed(7)
X, y = load_breast_cancer(return_X_y=True) # 569 samples, 30 features
X = StandardScaler().fit_transform(X)
T = y.reshape(-1, 1).astype(float)

L, C = 200, 1e-3
W = np.random.randn(30, L)
b = np.random.randn(1, L)
hidden = lambda x: 1 / (1 + np.exp(-(x @ W + b)))

# single machine
H = hidden(X)
beta1 = np.linalg.solve(np.eye(L)/C + H.T @ H, H.T @ T)

# split across K machines, each sends only its L x L and L x 1 partial sums
K = 10
A = np.zeros((L, L)); B = np.zeros((L, 1))
for block in np.array_split(np.arange(len(X)), K):
    Hk = hidden(X[block])
    A += Hk.T @ Hk
    B += Hk.T @ T[block]
beta2 = np.linalg.solve(np.eye(L)/C + A, B)

print("max difference between the two betas =", np.max(np.abs(beta1 - beta2)))
print("same to 1e-10 ?", np.allclose(beta1, beta2, atol=1e-10))
print("each machine sends one %dx%d matrix = %.2f MB, no matter how big N is"
      % (L, L, L*L*8/1e6))
```

Output:

```
max difference between the two betas = 2.445960101127298e-16
same to 1e-10 ? True
each machine sends one 200x200 matrix = 0.32 MB, no matter how big N is
```

The block-by-block β matches the single-machine β to about 10^{-16} — they are the same answer. This is the exact claim of Section 7, now demonstrated on real data.